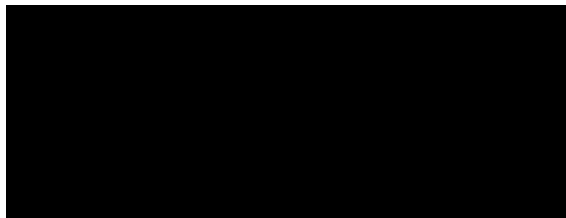


University of Waterloo
Faculty of Engineering
Department of Electrical and Computer Engineering

Swap-it – Electric Drone Battery Swapping System
Detailed Design and Project Timeline

Group 2017.032

Prepared By:



Consultant: David Wang

June 30, 2016

The overall score for this report was 65.9 / 80

Table of Contents

1	High Level Description of Project	2
1.1	Motivation.....	2
1.2	Project Objective.....	2
1.3	Block Diagram	2
1.3.1	Drone.....	3
1.3.2	PC.....	4
1.3.3	Assisted Landing System (ALS).....	4
1.3.4	Battery Swapping System (BSS)	4
1.3.5	Microcontroller Board	4
1.3.6	Motor Control Board (MCB).....	4
2	Project Specifications.....	4
2.1	Functional Specifications.....	4
2.2	Non-Functional Specifications.....	5
3	Detailed Design	6
3.1	Image Processing Application (IPA)	6
3.1.1	Details of Final Design	6
3.1.2	Engineering Design Process	10
3.2	Assisted Landing System (ALS).....	11
3.2.1	Details of Final Design	11
3.2.2	Engineering Design Process	15
3.3	Battery Management System (BMS).....	16
3.3.1	Hot Swap Circuit Design Process	17
3.3.2	Details of Final Charging Circuit Design	20
3.3.3	Details of Final Battery Compartment Design.....	21
3.3.4	Battery Compartment Design Process	22
3.4	Battery Swapping System (BSS)	23
3.4.1	Details of Final Design	23
3.4.2	Engineering Design Process	25
4	Discussion and Project Timeline.....	26
4.1	Evaluation of Final Design	26
4.2	Use of Advanced Knowledge	26
4.3	Creativity, Novelty, Elegance.....	27
4.3.1	Creativity.....	27
4.3.2	Novelty.....	27
4.3.3	Elegance.....	27
4.4	Student Hours.....	28
4.5	Potential Safety Hazards.....	28
4.6	Project Timeline.....	29
	References.....	30

1 High Level Description of Project

1.1 Motivation

More applications for autonomous vehicles (AVs) are developing every year. Drones can monitor forests for fires [1], robots can discover new planets [2], and much more. The fascinating thing about AVs, whether it be a land robot or a drone, is that they can perform various tasks without human intervention. Thus, the dangers and responsibilities a human would previously be exposed to are shifted to the machine. This shift allows humans to explore places not previously possible. However, a significant issue facing AVs today is their limited use due to a poor battery lifespan. Current AVs either shutdown on the spot, or carefully time their task(s) to ensure that there is sufficient battery life to return to an operator or charging station where the battery is exchanged manually or set to charge for several hours. Pertaining to electric drones specifically, the best ones on the market have an average flight time of only 20 minutes [3].

1.2 Project Objective

The objective of this project is to design a fully automated battery swapping station for electric drones. Swap-it aims to reduce the current battery limitations imposed on electric drones by providing a fully automated station where the drone can go to swap its depleted battery with a charged one (without losing power), allowing it to continue on with its duties in a matter of minutes. Swap-it can be used as a local or remote station, depending on the application.

1.3 Block Diagram

Swap-it is designed to detect the presence of an approaching drone and help guide its landing to a fixed location on the station. The swapping sequence will begin immediately after the drone has landed. Through a combination of digital control theory and precision mechanics, a microcontroller interfaces with a set of electric motors to precisely control the directional and rotational movements of the swapping system. The system extracts the drained battery from the drone and stores it for charging. It then retrieves a charged battery from the battery storage system and inserts it into the drone. The high-level system overview for Swap-it is shown in Figure 1 below.

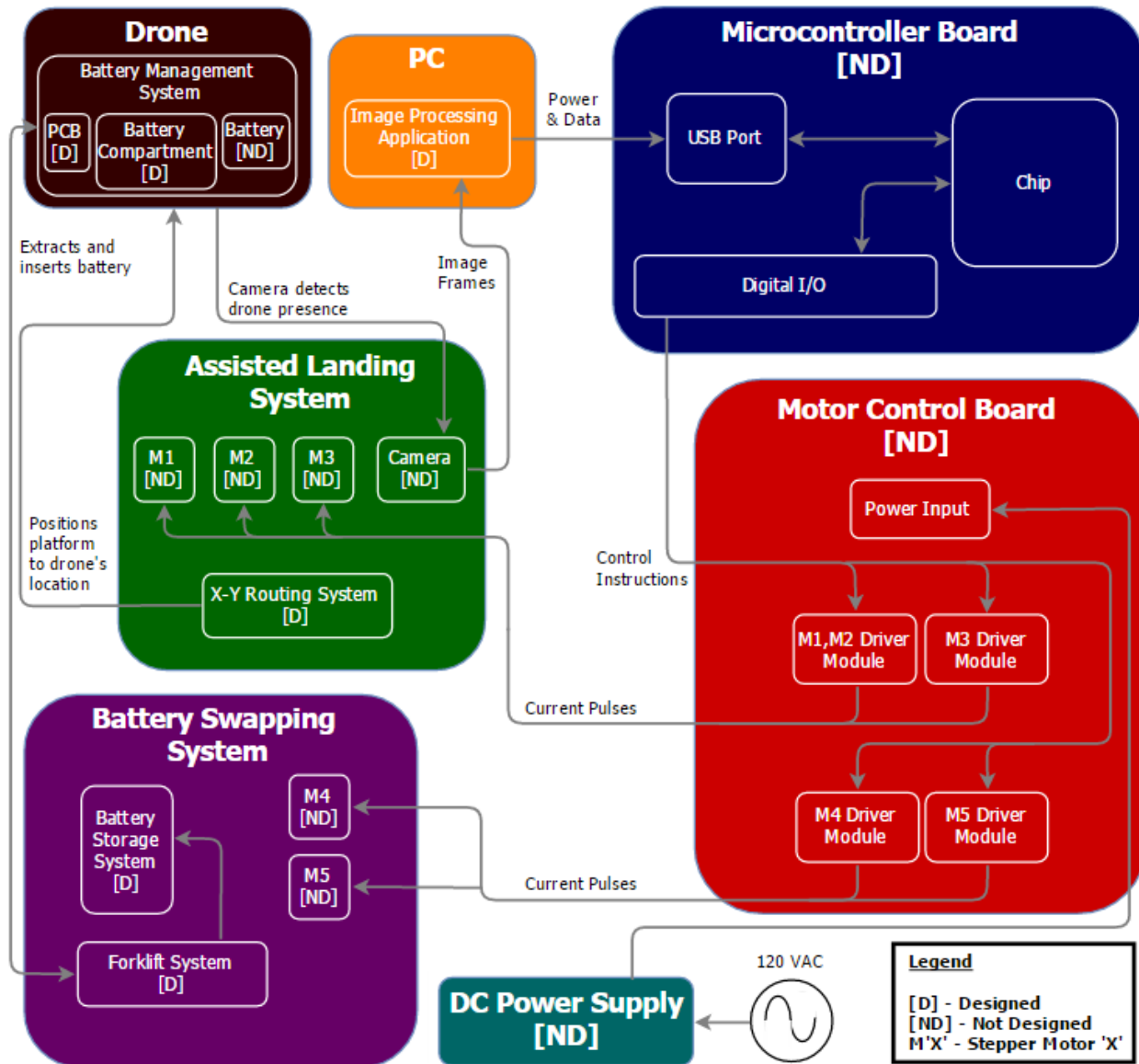


Figure 1. Block diagram showing the interconnectivities between subsystems.

1.3.1 Drone

The drone is equipped with a custom battery management system (BMS) located on its bottom that allows the battery to be removed and reinserted by the swapping station. The BMS consists of a printed circuit board (PCB), that's housed in a custom designed battery compartment, to maintain power to the drone during the battery swapping process.

1.3.2 PC

The role of the PC is to run the Image Processing Application, which is used to locate and track the drone during the landing process.

1.3.3 Assisted Landing System (ALS)

This subsystem consists of a moving platform with a sky-facing camera, driven by a set of three motors. The platform moves in the XY plane around the station to assist the drone's landing.

1.3.4 Battery Swapping System (BSS)

The BSS is a mechanical process driven by a set of two motors to remove, store, and replace the drone's battery.

1.3.5 Microcontroller Board

The Microcontroller board provides the logic that processes the incoming data from the Image Processing Application to control the ALS and the BSS.

1.3.6 Motor Control Board (MCB)

The MCB consists of a set of motor drivers that interface the commands from the Microcontroller Board to the individual motors.

2 Project Specifications

2.1 Functional Specifications

This section details the functional specifications for this design. They are broken down into two categories: essential and non-essential. Table 1 lists the essential specifications that must be met in order to consider this design complete. Table 2 lists the non-essential specifications that add value to the project, but are not required.

Table 1. Essential Functional Specifications

Subsystem	Specification	Description
Assisted Landing System	Landing Precision	The landing pad must be able to align itself under the center of the drone within a 2cm tolerance.
Assisted Landing System	Incoming Drone Speed	The ALS shall be able to land a drone with an incoming speed of 0.2m/s.
Image Processing Application	Drone Detection	The camera must be able to detect the incoming drone from a height of at least 1m above the station.
Image Processing Application	Tracking Latency	The application shall be able to start tracking the drone's motion within 200ms from the time of detection.
Battery Swapping System	Battery Swap	The drone's battery must be replaced with a new battery after landing on the platform.
Battery Swapping System	Battery Charging	Once removed from the drone, the drained battery shall be placed in a spot where it can be charged.
Drone	Drone Take Off	The drone must be able to continue flight after the battery has been swapped.
Battery Management System	Durability	The compartment must remain secure during regular drone operation until the drone has completely landed for batteries weighing up to 100 grams.
Microcontroller Board	Communication and Control	The Microcontroller must be able to receive control information through a serial port and process the logic required to control at least 3 motors simultaneously.

Table 2. Non-Essential Functional Specifications

Subsystem	Specification	Description
Battery Management System	Power Loss	The drone should not lose power during the battery swapping process.
Battery Swapping System	Consecutive Battery Swap	The station shall be able perform consecutive battery swaps without human intervention.

2.2 Non-Functional Specifications

In addition to functional specifications, this project also contains non-functional specifications. Table 3 lists the essential non-functional specifications, and Table 4 lists the non-essential non-functional specifications.

Table 3. Essential Non-Functional Specifications

Subsystem	Specification	Description
Assisted Landing System	Size of Landing Area	The dimensions of the landing area shall be at least 50cm x 50cm.
Drone	Drone Payload	The BMS added to the drone must be less than 1500g.
Power Supply	Supply Voltage/ Current	The system shall run off a 12V DC power supply and not consume more than 150W.

Table 4. Non-Essential Non-Functional Specifications

Subsystem	Specification	Description
Overall System	Total Size	The total size of the station shall not exceed 1m x 1m.
Overall System	Total Mass	The total mass of the station shall not exceed 15kg.

3 Detailed Design

This section describes the designs and design processes of Swap-it's four core subsystems.

3.1 Image Processing Application (IPA)

The purpose of the IPA is to assist in identifying and tracking the drone. From a high level viewpoint, the system must be capable of assisting the drone in landing accurately with an error margin no greater than 2 cm. To accomplish this, the IPA takes as input the live video stream, with the camera positioned on the landing pad facing up. The application is always running and monitoring for a drone to enter the camera's field of view. Once a drone enters, the application uses blob detection to identify the position of the drone. After identifying the position, the application translates this information into coordinates, sending them to the microcontroller over a serial communication line to be further processed as instructions for the Assisted Landing System (ALS).

3.1.1 Details of Final Design

To identify the drone, we use a blob detection algorithm built on the OpenCV library [4] that looks for a circular sticker with a distinct green color that's centered on the bottom of the drone. Blob detection is a technique within the field of computer vision that identifies regions within an image based on the contrast of a particular image property. The blob is generally consistent in terms of the measured property, making it identifiable as a single entity. We have chosen to do blob detection because it effectively identifies the drone while preserving simplicity and performance. Given that we designed a relatively unique appearance for the bottom of the drone, this eliminates a lot of the complexity in the feature detection algorithm by having something fairly distinct to identify. With this approach, using colour as the measured property produces accurate results meeting our Tracking Latency and Drone Detection specifications. The IPA takes input from the camera through direct connection by USB. The information

is processed and passed through a serial port to the microcontroller board which controls the ALS. We utilized some online resources [5] [6] [7] to help build the main algorithm that runs the application. Figure 2 below shows a high level overview of the transformation that a raw image frame from the camera goes through to produce drone location information for the ALS subsystem.

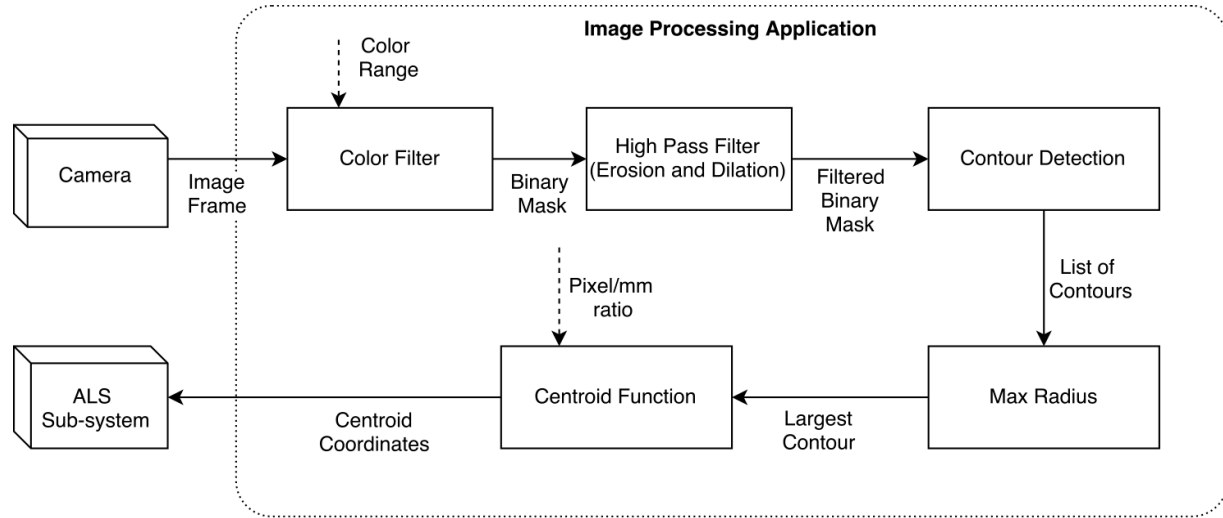


Figure 2. Overview of IPA sequence.

Dotted arrows show configuration parameters that calibrate the system while solid arrows show the flow of data from one block to the next. Below is a short description of the transformation each logical block performs on its input and the output it produces:

1. A frame is read from the camera.
2. A binary (black and white) mask is produced by applying a colour filter to the original frame. This filter transforms each pixel in the input image based on a predetermined range of colours, producing a white pixel if the pixel falls within the range and a black pixel otherwise.
3. The mask is further developed by applying a high pass filter. Erosion is a filter that makes the boundaries of a foreground object smaller. This process eliminates white noise around the object boundaries. However, this reduces the size of the object so a dilation filter is applied afterwards to increase the size back to its original value. [7]
4. Contour detection then identifies all contours on the frame according to the mask. Contours are edges in the image that make a closed loop. These usually identify physical objects.
5. The largest of the contours is selected according to the radius. This is likely the object we would like to track since it is the largest component of the frame which matches the desired features.
6. Given that the radius is significantly large, the object is worth tracking, and the centroid of the contour is calculated and converted from pixel dimensions to millimeter coordinates to be used by the ALS.

Most of the logical blocks above can be mapped to concrete functions as shown in the code snippet in Figure 3 below.


```

def send_coordinates(serial, coordinates):
    command = "X0 Y0 {} {} \n".format(*coordinates)
    serial.write(command)

def find_biggest_contour(frame):
    contours = find_contours(frame)
    if len(contours) > 0: # drone is present
        max_contour = max(contours, key=cv2.contourArea)
        (_, radius) = cv2.minEnclosingCircle(max_contour)
        moments = cv2.moments(max_contour)
        drone_centroid = (int(moments["m10"] / moments["m00"]), int(moments["m01"] /
moments["m00"]))
        return (radius, drone_centroid)
    return None, None

def find_contours(frame):
    return cv2.findContours(frame.copy(), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)[-2]

def colour_filter(frame, colour_lower, colour_upper):
    hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv_frame, colour_lower, colour_upper)
    return mask

def reduce_noise(frame):
    kernel = numpy.ones((3,3),np.uint8)
    mask = cv2.erode(frame, kernel, iterations=3)
    mask = cv2.dilate(mask, kernel, iterations=3)
    return mask

def centroid_to_coordinates(centroid):
    x_coord = pixels_to_mm(centroid[0] - REFERENCE[0])
    y_coord = pixels_to_mm(centroid[1] - REFERENCE[1])
    return (x_coord, y_coord)

def process_next_frame():
    (grabbed, frame) = camera.read()
    colour_mask = colour_filter(frame, COLOUR_LOWER, COLOUR_UPPER)
    filtered_mask = reduce_noise(colour_mask)
    (radius, drone_centroid) = find_biggest_contour(filtered_mask)
    if radius > 10:
        coordinates = centroid_to_coordinates(drone_centroid)
    send_coordinates(ser, coordinates)

```

Figure 3. IPA Code snippet.

To justify that we meet the tracking latency specification we measure the exact time taken by the IPA and each logical block in the algorithm using the flame graph analysis technique [8]. In this analysis we profile the stack frame of the program every millisecond and plot the results on a graph that depicts the relative amount of time taken in each logical block. Each nested function call is stacked on top of the calling function, showing exactly how much time that function call took to return. Additionally, we measure the average time taken for processing each frame using a simple benchmarking application that reads similar to the code shown in Figure 4 below. This analysis gives us the ability to identify bottlenecks and optimize our performance to meet the Tracking Latency specification.

```

start_time = time.time() # current time
# grab frame and process it
elapsed_time = time.time() - start_time

```

Figure 4. Benchmarking code snippet.

Figure 5 below shows the flame graph for our design. Even though we use a camera with a specification of 30 frames per second, we found that we cannot read frames at that rate. The practical performance is approximately 10 frames per second for the camera. A higher rate would require a camera that performs compression before transmitting its data on the USB link or adding complexity to our application by separating the I/O to a separate thread or process. Since time for completing a prototype of the platform is limited, we keep the algorithm simple and read frames on the main thread without compromising our Tracking Latency specification. We achieve the desired specification with a large margin since the average times taken for grabbing a frame and running our blob detection algorithm are 102ms and 18ms respectively, calculated using the benchmarking code shown in Figure 4 above. The time taken to send coordinates over the serial link is negligible.

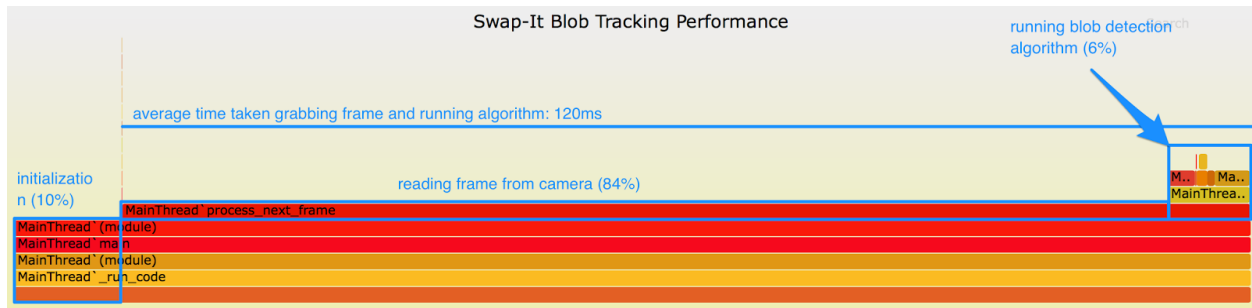


Figure 5. Blob tracking Flame Graph.

To justify the Drone Detection specification, we calibrate the minimum contour radius so that the drone is identifiable from the specified height. A circular sticker, 5 cm in diameter, is bound to the bottom of the drone. Through a simple experiment, shown in Figure 6 below, we found this sticker to appear with a diameter of 30 pixels in the video stream when it was held at a height of 1 meter above the camera.

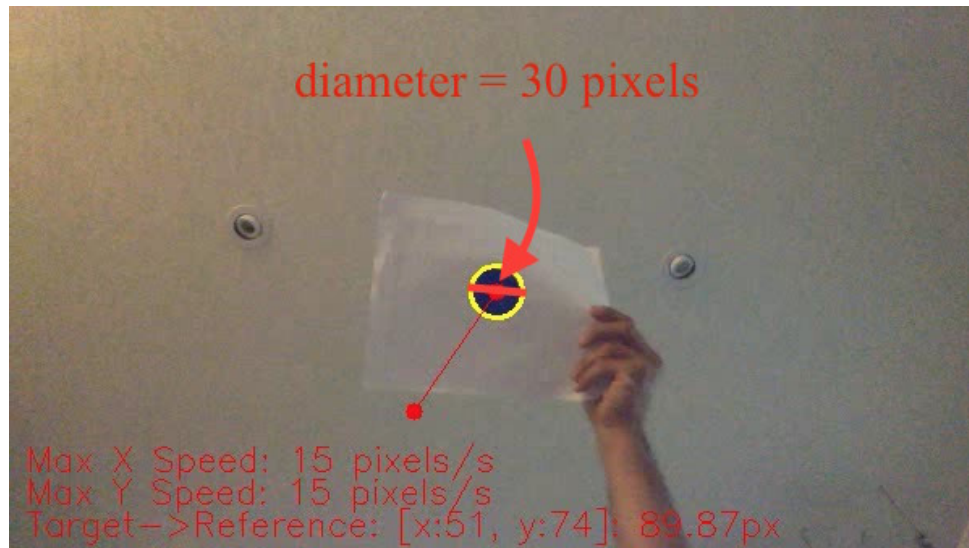


Figure 6. Image of blob detection experiment.

To allow for both the detection of the drone at a height of at least 1 meter and ignoring noise at much larger heights, we chose a minimum contour radius of 10 pixels.

3.1.2 Engineering Design Process

In addition to blob detection, we also considered other appearance-based object detection algorithms such as edge matching and corner detection. Both of these approaches involve identifying the shape of the drone based on either edges or corners. Edge matching would require our algorithm to have a unique template for every recognized drone. This quickly becomes a limiting factor in performance, as the algorithm would need to check the edges of an image against a set of templates thus compromising the scalability of our algorithm as the number of hosted drones increase. Accurate corner detection requires modification of the outline of user's drone to comply with a unique corner pattern, which is not desirable considering the added complexity and reduced compatibility. We find both these approaches to be limiting, given that as a product we would like our system to be both expandable and scalable for a variety of drones. Another limitation to these techniques is the need for a detailed or complex template to ensure accurate identification. Low accuracy would imply objects or disturbances in the environment, like birds, may be incorrectly marked as a drone. This is undesired behaviour and the complexities of detailed templates are easily avoided using blob detection.

We also considered a motion tracking approach in addition to appearance based approaches. There are several known algorithms such as mean-shift and optical flow for motion tracking. Generally, these algorithms measure differences between multiple frames as opposed to a single frame like blob detection. An advantage to using motion tracking is having more information regarding the flight of the drone. This could provide measurements such as speed and direction that could be used to better identify the flight pattern of the drone. Having an understanding of the drone's flight properties allows us to predict the future position of the drone and adjust the platform accordingly. Ultimately, we had hoped this would

increase how tightly coupled the movements of the platform and the drone were. However, this proves to be more complex than necessary, while the performance benefits are negligible. Also, a major problem with motion tracking is that it identifies multiple objects, and it is difficult to constrict it to our drone without additional appearance-based algorithms. In the end, blob identification using colour proves to return the best results while also having the simplest and most effective implementation for Swap-It.

3.2 Assisted Landing System (ALS)

The ALS is the system responsible for detecting and tracking the drone's location as it approaches the station, ensuring a successful and precise landing. The design of the ALS can be broken down into a mechanical and a software component.

3.2.1 Details of Final Design

The ALS follows the incoming drone according to the tracking information provided by the Image Processing Application (IPA). The input is transmitted over a USB serial communication stream from the PC running the IPA. The microcontroller receives a coordinate specifying where the drone is relative to the landing platform. The ALS then drives the platform to the coordinate of the drone while continuously processing new incoming coordinates. The target location of the platform is continuously updated until the drone has landed. The flowchart in Figure 7 below shows the flow of data through the ALS.

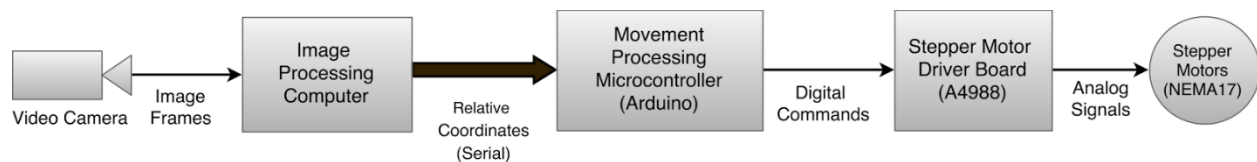


Figure 7. Flowchart of ALS sequence.

The ALS is a complex electromechanical system comprised of the parts shown in Table 5 below. We are using Autodesk® Fusion 360™ [9] to design custom parts that we are printing using a 3D printer.

Table 5. ALS Parts list.

Part	Qty.	Designed	Part	Qty.	Designed
Stepper Motor	3	N	Bearings	2	N
20 Tooth Pulley	1	N	Camera	1	N
40 Tooth Pulley	2	N	Belt	3	N
GT2 Belt	2	N	Y-Axis Motor Holder	2	Y
8mm Linear Shaft	2	N	Y-Axis Bearing Holder	2	Y
12mm Linear Shaft	2	N	Bearing Cover	2	Y
8mm Bushing	4	N	Y-Axis Carriage	2	Y
12mm Bushing	4	N	X-Axis Motor Holder	1	Y
Arduino Mega 2560	1	N	X-Axis Bearing Holder	1	Y
RAMPS 1.4 Controller	1	N	X-Axis Carriage	1	Y
Stepper Motor Driver	2	N	Platform Base	1	Y

After assembly, the ALS looks like the model shown in Figure 8 below. The landing pad mounts on top of the X-Axis Carriage and can be seen in the complete model shown in Figure 20 in the BSS section.

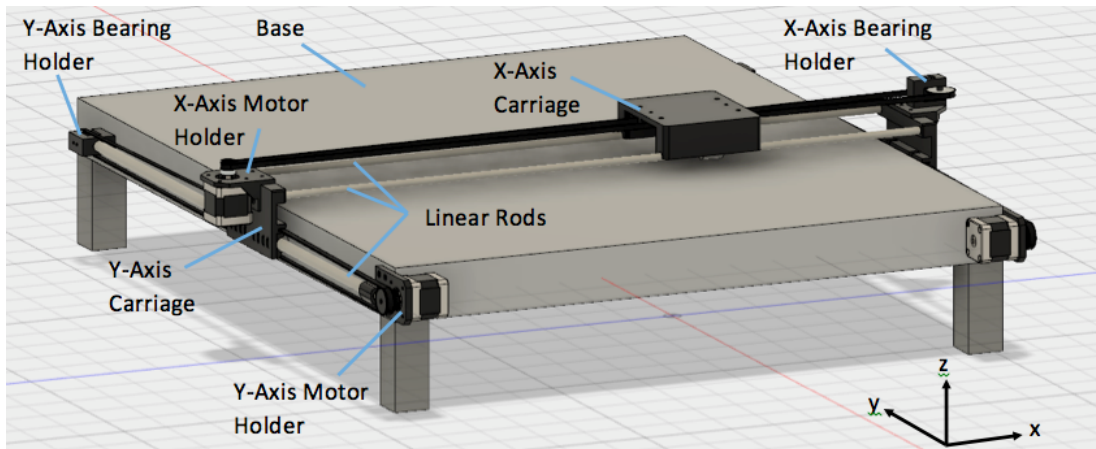


Figure 8. 3D Model of ALS. [10] [11]

Table 6 below summarizes how we met the specs related to the ALS.

Table 6. ALS Specification summary.

Specification	Description	Justification
Landing Precision	The landing pad must be able to align itself under the center of the drone within a 2cm tolerance.	Stepper motors have 1.8-degree increments per step. Therefore (assuming 1/8 micro stepping), $Precision = \frac{mm}{step} = \frac{(belt\ pitch) * (pulley\ number\ of\ teeth)}{(motor\ steps\ per\ rev) * (driver\ microstep)}$ $= \frac{(2mm) * (40)}{\left(\frac{360}{1.8}\right)(8)} = 0.05\ mm/step$ where $0.05\ mm \ll 2\ cm$.
Incoming Drone Speed	The ALS shall be able to land a drone with an incoming speed of 0.2m/s.	The maximum motor speed is approximately 1000 rpm. The landing pad can move in both directions at the same time, therefore both axes must be considered. <i>Y – axis pulley circumference</i> $= (\#\ of\ teeth) * (tooth\ size)$ $= 40 * 2mm = 80mm = 0.08m$ <i>X – axis pulley circumference</i> $= (\#\ of\ teeth) * (tooth\ size)$ $= 20 * 2mm = 40mm = 0.04m$ X-axis speed in our system is, $= (rpm) * \frac{(.04m)}{\frac{60s}{1min}} = \frac{1000 * 0.04m}{60} = 0.67m/s$ and the Y-axis speed is double, which yields 1.33 m/s. Both speeds are much greater than 0.2 m/s.
Size of Landing Area	The dimensions of the landing area shall be at least 50cm x 50cm.	We mounted the linear rods on the side of the platform to maximize the area on top, as opposed to a complete tabletop solution. Both the X-Axis and Y-Axis linear systems were made 75cm in length. With X-carriage measuring 11cmx14cm, this limits some the movable distance of the platform. $Landing\ Area = (75cm - 11cm)(75cm - 14cm)$ $= 64cm \times 61cm$ This exceeds the specification.

The brain of the ALS is the microcontroller board. This board is responsible for processing coordinates coming from the IPA and converting them into instructions for the motor. The firmware is loaded into FLASH memory on the board. The firmware utilizes an open source library [12] to help communicate

with the motors. Inputs are marshalled by the library and passed to the motor driver to be converted into analog signals that drive the stepper motors.

The ALS has 6 total states that it can be in, as shown in Figure 9 below. During initialization and once the swapping operation has completed, the system is in the homing state where it ignores all input, the landing pad returns to its home position, and prepares for the next landing. Once in the home position, the ALS transitions into the idle state and begins accepting commands on its serial port. Upon receiving a coordinate from the IPA, it starts moving towards the drone and tracking its location based on the inputs it receives. Eventually, the drone lands and a signal is sent to the microcontroller. At this point, the ALS becomes responsible for moving the drone to the BSS and ignores all further input until the next time it is in the idle state. The BSS performs the swap and the drone takes off. The platform then returns to the homing state and prepares for the next landing. Figure 9 below shows the states of the ALS process.

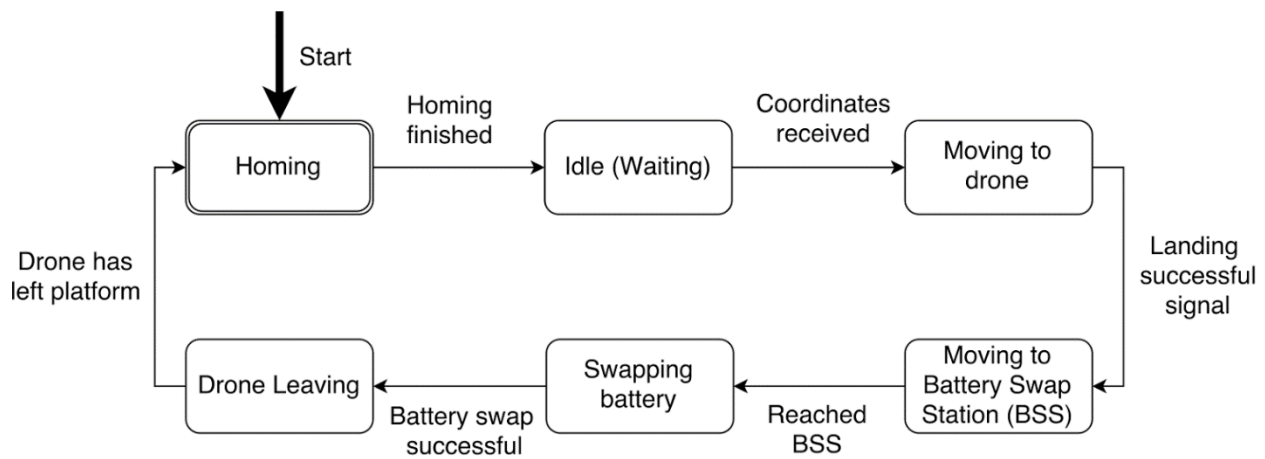


Figure 9. State diagram of the ALS process.

Two components exist in the firmware design of the ALS. The first component is the routing framework that was designed to not block the program loop, allowing near real-time updates to the movement targets. The second component is a fast communication pipeline implemented based on the design of the Marlin [13], which is prevalent amongst 3D printers. Essentially, a buffer of four commands is kept in memory and executed one after another in every iteration of the main program loop.

3.2.1.1 ALS Routing Framework

The ALS routing framework is built on top of an open-source driver [12] for the motor driver boards. Figure 10 below shows an overview of the framework.

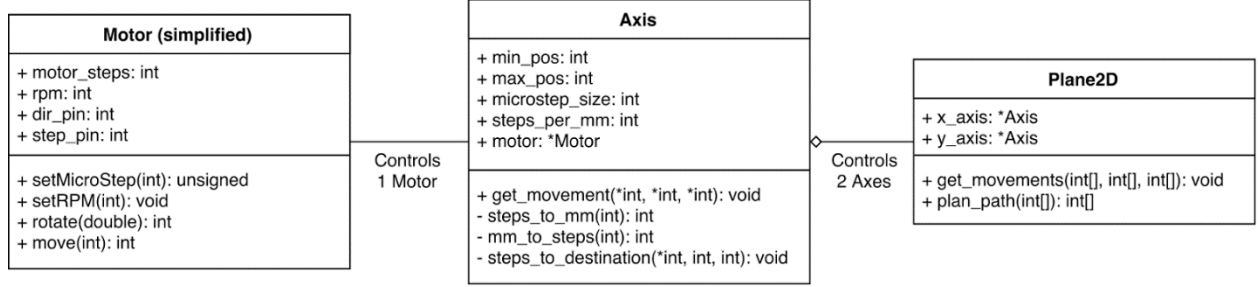


Figure 10. UML diagram of ALS routing framework.

The Motor class is provided by the StepperDriver library [12] and is responsible for sending control information to the motor driver boards based on how the class' instance is configured. The Axis class is responsible for controlling the movement of a single motor within the boundaries defined by the user. Additionally, it encapsulates the logic for converting millimeters to steps once it has been initialized with number of steps per millimeter. This ensures that all public API defined for the Axis class can receive millimeter as its units. The Plane2D class controls 2 axes to create a full routing system. Its primary function is simply controlling the Axis objects but can also plan a path that avoids any user-defined obstacle.

To justify that our firmware can provide the logic to meet the Landing Precision specification, we estimated how far the landing pad can lag the drone's movement due to delays in the running code with the assumption that the IPA provides the ALS with perfect information regarding the drone's coordinates. The maximum speed at which the drone can travel in any direction is 0.2 m/s according to the Incoming Drone Speed specification. Note that Table 6 already justifies the motors' physical capabilities to meet the spec.

We already know that $Max(V_{Drone}) = 0.2 \frac{m}{s} = 0.2 \frac{mm}{ms}$ from the specifications. We experimentally found that our software cannot reach the maximum speed mentioned in Table 6 because the cost of writing to the microcontroller's digital I/O pins reduces the number of steps we can make per unit of time. Experimentally, a value of $Max(V_{axis}) = 0.54 \frac{m}{s} = 0.54 \frac{mm}{ms}$ was obtained. Dividing this rate in half to account for driving the second axis on the plane, we have an overall rate of $Max(V_{overall}) = 0.27 \frac{mm}{ms}$ in any direction. We have a margin of $0.07 \frac{mm}{ms}$ according to this estimation, which meets the specification. Note that this calculation was done using the native I/O library. There exist libraries such as DirectIO that are able to give us huge performance boosts by reducing the cost of I/O if the need arises [14].

3.2.2 Engineering Design Process

The first step in the design was determining how to move the landing pad around the base. While doing so, we needed to bear in mind the dependencies of the other connected systems to ensure proper cohesion. We considered two different systems: S1- the x-y linear rod system, controlled by motors fixed to the base; the other alternative was S2- the omnidirectional pad on wheels, driven by motors and wheels

attached to the moving pad, controlled by RF commands from the base. As accuracy is the most important requirement for the ALS, it was very clear that S1 was the right choice. S1 avoids the risks of wireless communication latency and error, reduces complexity by limiting movement to two directions, and provides accurate coordinates and location feedback.

After determining that the x-y linear rod system was the best choice, we needed to size the motors, rails, and the other standard mechanical components. Then we designed the custom printed parts to tie everything together. There was not much flexibility in the selection of most parts. However, selection of the motors was crucial. There are various types of motors that can be used for our application, each offering advantages in certain categories and disadvantages in others. The key factors in the motor selection process are precision, compatibility, torque, and speed. We decided to use stepper motors, as they had the highest rankings in precision and compatibility which are the two most important factors for our application.

Once the mechanical design was configured, the software/firmware design process began. Very early in the prototyping phase, we found that instantiating new objects or reallocating memory in the main program loop greatly impacted the performance of the ALS. For example, at first the `get_movement` method of the `Axis` class created a new variable with the number of necessary steps the motor should move. This turned out to be the cause of a major slowdown in the program loop. To circumvent this issue, we instead pass pointers to integers to the function, and the function fills the memory address pointed to by the pointer with the correct number of steps. This is the first design principle that is honored throughout the rest of the software design.

A movement unit is the maximum number of steps that each stepper motor is allowed to move per iteration of the program loop. Another issue that we encountered early on was when we used a longer movement unit for each motor. Since the motors need a slight delay to build up their momentum, the longer movement unit gained more speed. However, we soon realized that using a long movement unit blocked the software for too long. This caused the landing pad to lag behind the drone's movement. Hence, another principle reflected into the ALS routing framework is a reasonable upper bound on the movement unit size. The bound guarantees that we do not lose control over the firmware's operation while still meeting the specifications for the ALS.

3.3 Battery Management System (BMS)

The purpose of the battery management system (BMS) is to allow the drone's battery to be easily removed, and to ensure that power to the drone is maintained during the entire battery swap process. There are two specifications related to the BMS section. The first is that the battery compartment must stay secure during flight and landing, for a weight up to 100g. The second specification states that the drone should never lose power while its battery is being swapped. To accomplish this, the external battery for the drone will be enclosed in a plastic case, which is held onto the drone via permanent magnets. To complete the hot swap, power for the drone is sourced from one of two locations: a small non-removable (internal) battery or a large removable (external) battery. The large battery supplies voltage when it is present, and once it is removed, the small battery powers the drone. The larger battery charges the non-

removable battery when it is attached so that there is always a redundant power supply. Figure 11 below shows a block diagram for the BMS and its interface with the drone.

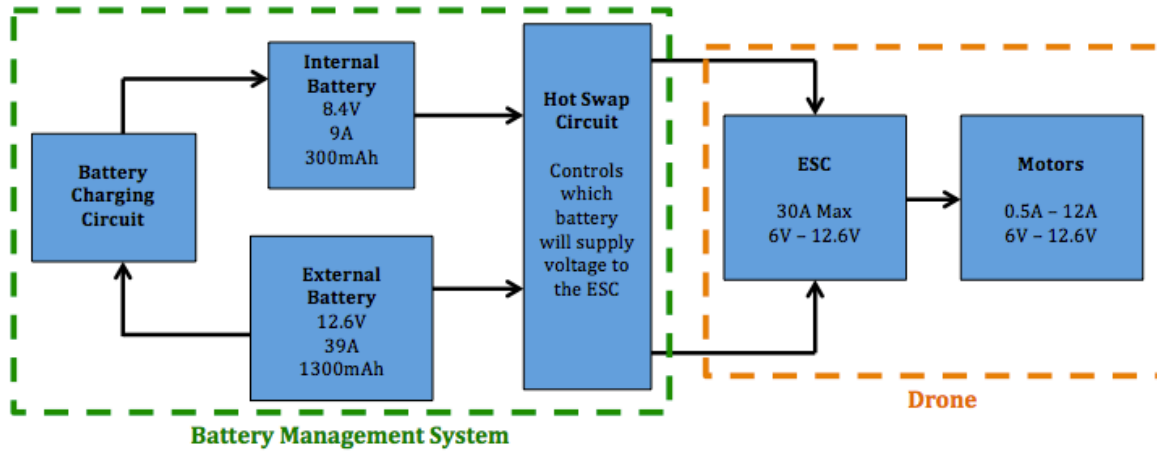


Figure 11. Block Diagram of the BMS.

The drone specification states that it must be powered by a 30C, 2 or 3-cell lithium-polymer battery, which results in a voltage range of 6V-12.6V [15]. For the large battery, we chose to use 3-cells with a capacity of 1300mAh so that the drone can fly for 6.5mins at maximum speed. The internal battery only supplies power when the drone has landed on the Swap-It platform. A 2-cell, 300mAh battery is used to minimize the weight, but still be able to source 9A for 2 minutes, if needed.

3.3.1 Hot Swap Circuit Design Process

The goal of the hot swap circuit is to control which battery supplies power to the drone at any given time. Two different methods to accomplish this have been designed. While only one design can be used in the project, by designing two circuits we were able to choose the best option. The first method uses a comparator with a N-channel MOSFET, and the second uses two ideal diode integrated circuits.

3.3.1.1 Hot Swap Circuit Design #1

First, the design using a comparator and an N-channel MOSFET will be discussed. This circuit controls the output voltage, V_{drone} , by comparing the internal and external battery voltages. Figure 12 below shows the first scenario when both batteries are connected. The inverting terminal has a higher potential than the non-inverting terminal, so the output voltage of the comparator is low. The MOSFET is connected in a common-drain configuration and used as a switch. Since the input to the MOSFET is zero, the transistor is off and this circuit does not supply voltage to the drone. The drone will receive power directly from the external battery.

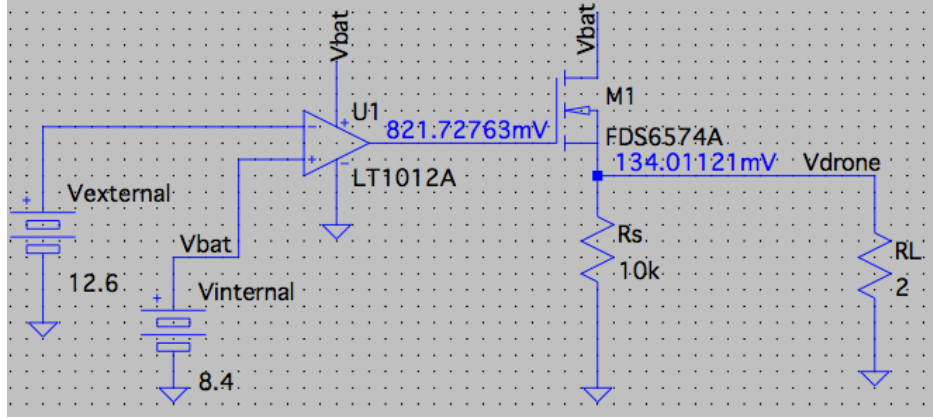


Figure 12. Hot Swap Circuit #1 when external battery is present.

Figure 13 below shows the second scenario, when the external battery is removed. Now the voltage on the non-inverting terminal is greater than the inverting terminal, and the output of the comparator is high. This voltage drives the transistor and the output of the MOSFET supplies power to the drone. The choice of MOSFET in this design is critical. Specifically, the MOSFET must have an appropriate thermal voltage, power rating, and on resistance to output an acceptable voltage and source enough current. We choose to use FDS6574A, which is manufactured by Fairchild and in stock on Digikey. When connected in a common-drain configuration, the MOSFET has the following voltage transfer equation:

$$\frac{V_{out}}{V_{in}} = \frac{gmR_s}{1 + gmR_s} \approx 1, \text{ when } R_s \gg 1$$

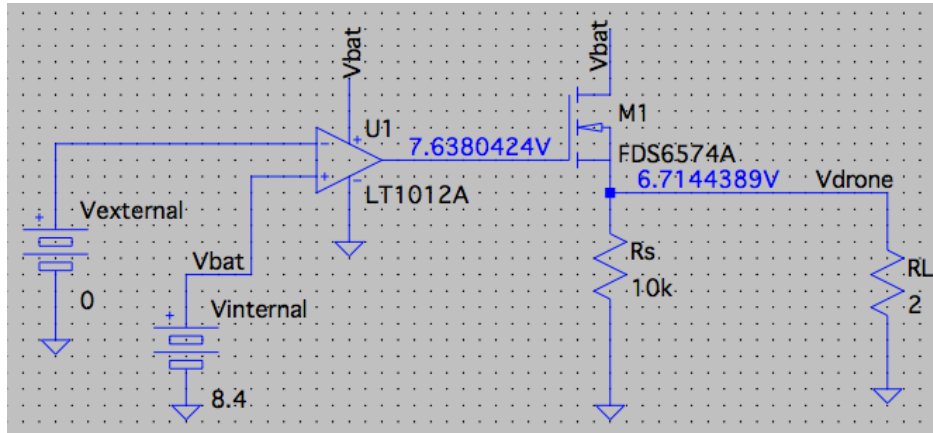


Figure 13. Hot Swap Circuit #1 when external battery is removed.

The threshold resistance is critical because it determines the maximum voltage at the drain of the device. For this application, we want the output voltage to be identical to the input voltage. This means the threshold voltage must be as small as possible, while still maintaining the ability to successfully turn off

the transistor. This device's threshold voltage is 0.65V [16], which gives an output voltage of 6.7V when the MOSFET is on.

We expect that the maximum current through this device will be 4A, since the drone is not flying [17]. The total power dissipation in a MOSFET is composed of switching power and conduction power. The switching power in this application is negligible since the MOSFET only switches the drone's battery is swapped. From the data sheet of this device, the maximum power dissipation is 2.5W, with adequate heat sinking. Power dissipation depends on the current through the device and the on resistance of the MOSFET. To minimize power dissipation, we choose a MOSFET with a small on resistance of 7mΩ [16]. The following calculation shows that this device can supply 4A to the drone without overheating:

$$P_{on} = R_{DS(on)} \times I_{on}^2 = (7m\Omega)(4A)^2 \rightarrow P_{on} = 28mW \ll 2.5W$$

3.3.1.2 Hot Swap Circuit Design #2

The second design for the hot swap circuit uses ideal diodes to choose the power supply with the higher voltage. A small IC (LTC4352), manufactured by Linear Technology, was chosen to simulate the behavior of ideal diodes. The use of typical diodes with a 0.7V voltage drop would not be sufficient in this design because 8.3% of the battery would be dropped across the diode. Schottky diodes were considered, but would still drop 3.6% of the battery voltage across them. The ideal diode IC only drops 0.31% of the 8.4V battery across it. This particular chip was chosen because it has a voltage rating of 18V [18], which surpasses the requirements for this design. The MOSFET used in this design is the same device as previously described. Figure 14 and Figure 15 below show the circuit when both batteries are connected, and when the external battery is removed, respectively.

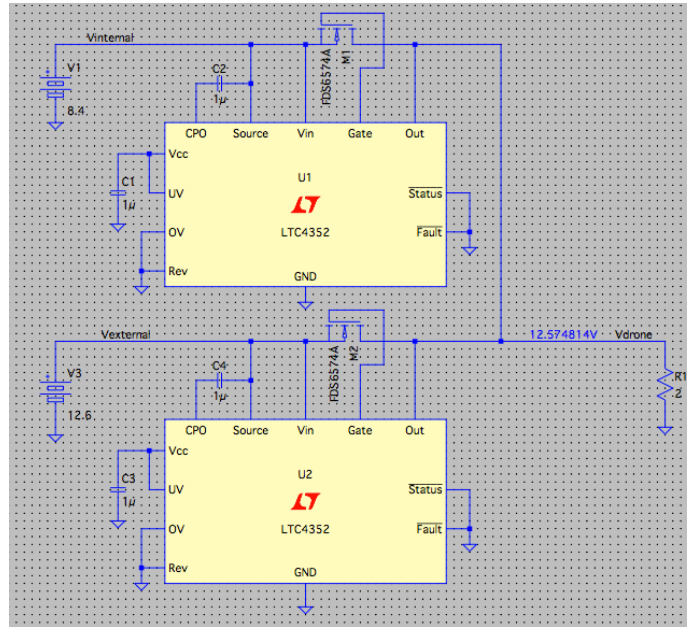


Figure 14. Hot Swap Circuit #2 when external battery is present.

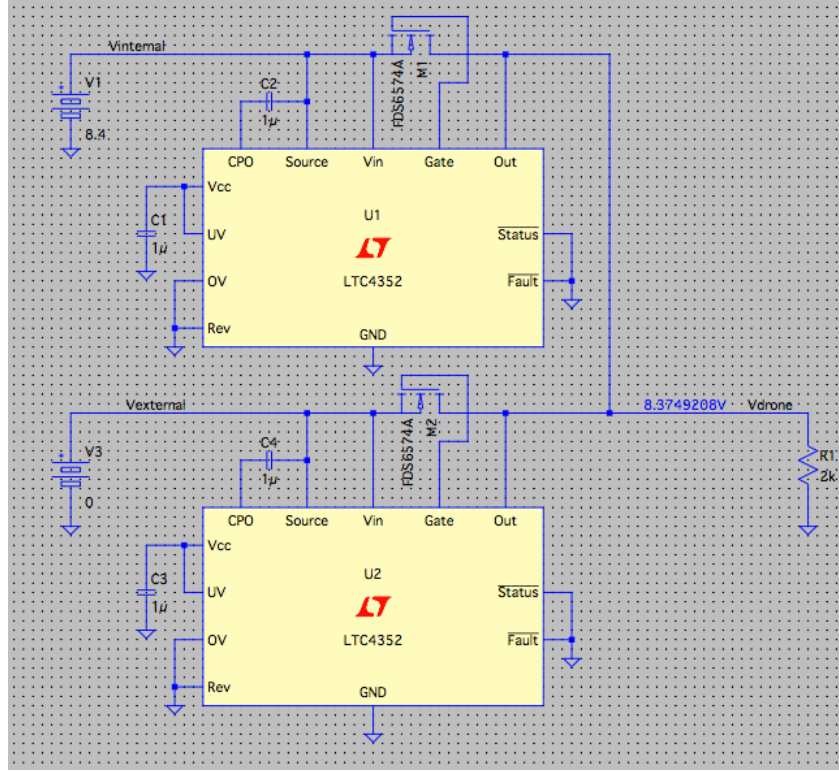


Figure 15. Hot Swap Circuit #2 when external battery is removed.

3.3.1.3 Hot Swap Design Conclusion

For this project, the hot swap circuit with ideal diodes is used. The main reason this option was chosen is because the output voltage is much closer to the input (desired) voltage. In the first method, a total of 1.7V is lost in the comparator and MOSFET. Both methods meet our specifications, but it is clear that using ideal diodes will simplify the design and make efficient use of the batteries.

3.3.2 Details of Final Charging Circuit Design

For the hot swap circuit to function, it is important that the internal battery always has sufficient charge. When the removable battery is connected, the charging circuit uses its power to charge the internal battery. This circuit will consist of the charging IC (BQ2057WSN) and its recommended peripheral components. This particular chip was chosen because it is capable of charging and balancing a two-cell LiPo battery, and uses three-phase charging [19]. Figure 16 below shows a sample design, taken from the data sheet for this device. The final circuit will mimic this design.

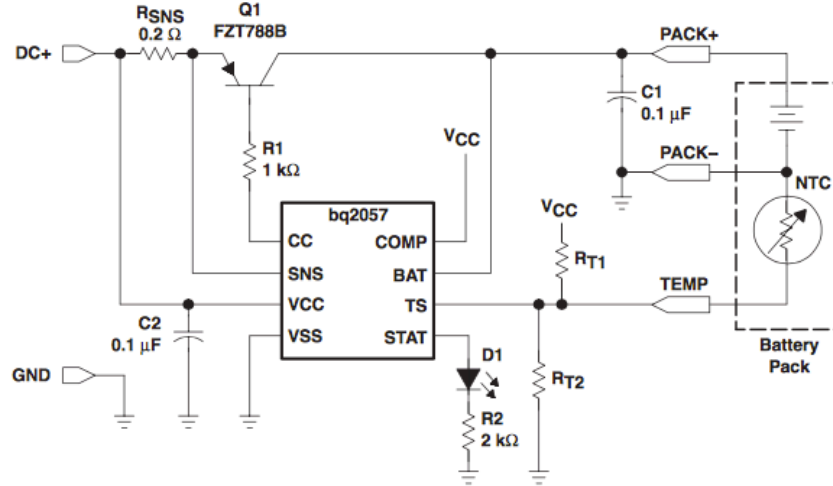


Figure 16. Design for Charging Circuit.

3.3.3 Details of Final Battery Compartment Design

The main function of the battery compartment is to allow easy removal of the external battery, while also maintaining a secure connection when the drone is in flight and during landing. To accomplish this, we designed a battery enclosure that connects magnetically to the drone. Figure 17 below shows an assembled diagram of the battery compartment, and Figure 18 below depicts an exploded view. Rare earth magnets sit in each corner of the removable battery, and also in the compartment that houses the internal battery and PCB. When the removable battery is placed close to the opposite magnets, it snaps into place. Rare earth magnets have a force of 30N [20], which corresponds to a pound-force of 6.7, well beyond the specification of 100g.

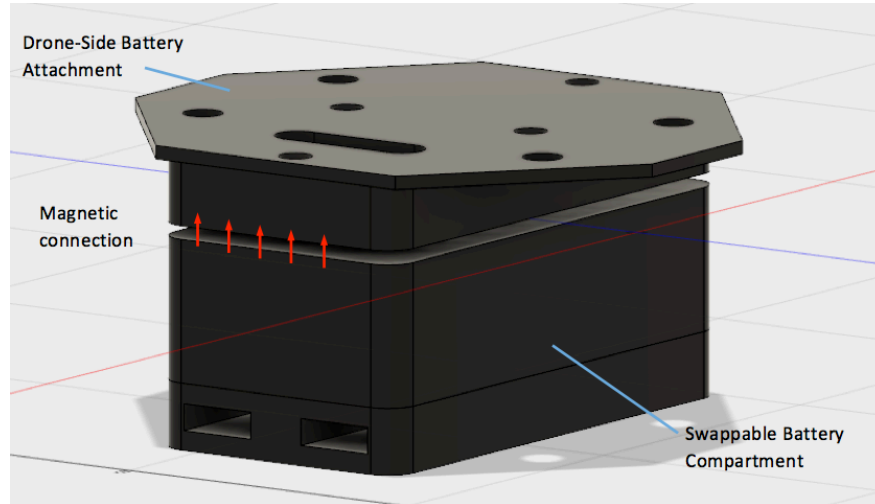


Figure 17. Assembled Diagram of the Battery Compartment.

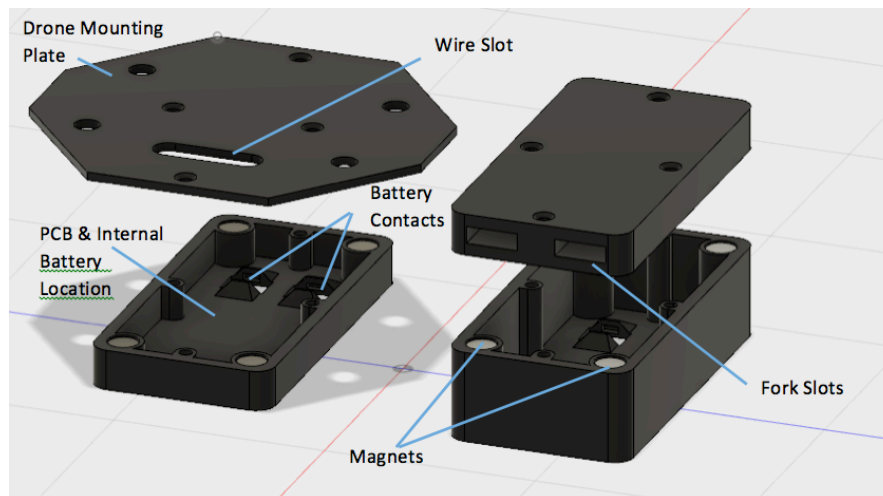


Figure 18. Component Diagram of the Battery Compartment.

3.3.4 Battery Compartment Design Process

There are many ways to secure the battery compartment to the drone. One problem we identified early on was that it would be important to make sure that the battery contacts lined up perfectly each time it was connected and the battery still remained easy to remove. To overcome this problem, we decided to use magnets as the primary force attaching the battery to the drone. Rare earth magnets are very strong, and have the added benefit of auto-aligning the contacts.

One problem with rare earth magnets is that they cannot turn themselves off to easily disconnect the battery. A proposed solution to solve this was using electromagnets to generate a magnetic field in the opposite direction of the permanent magnet's field to cancel it. We tested this design hypothesis by making an electromagnet and placing it next to the permanent magnets. We used an iron core with 4900 turns of 28 gauge copper wire. The solenoid was powered with an 18V battery, and the wire had a total

resistance of 98ohms [21]. This means that 180mA of current was travelling through the solenoid. A rare earth magnet typically has a magnetic field of 1.4T [22]. The following calculation proves that with 4900 turns it is not possible to overcome a magnetic field of 1.4T.

$$B = \mu NI = (200)(12.56 \times 10^{-7} H)(4900)(180mA) = 0.22T$$

Our experiment confirmed this result. In order to create a magnetic field of 1.4T, the solenoid would require 31000 turns, assuming the current through the solenoid did not change. The electromagnet would be placed on the drone, which is not capable of supporting the weight of a coil with anywhere close to 31000 turns.

3.4 Battery Swapping System (BSS)

The BSS is responsible for removing and storing the drained battery from the drone, then retrieving and reinserting the charged battery.

3.4.1 Details of Final Design

Coming up with a reliable and elegant design for the BSS has been the most challenging part of this project thus far. The design we ended up selecting resembles that of a forklift system. The majority of the parts that make up the BSS are 3D printed parts that we designed in Autodesk® Fusion 360™ [9]. Table 7 below lists the parts used in the BSS.

Table 7. BSS Part Listing.

Part	Qty.	Designed	Part	Qty.	Designed
Stepper Motor	2	N	Arduino Mega 2560	1	N
Stepper Motor Driver	2	N	RAMPS 1.4 Controller	1	N
20 Tooth Pulley	1	N	Bearings	1	N
8mm Lead Screw	1	N	Y-Axis Motor Holder	1	Y
GT2 Belt	1	N	Y-Axis Bearing Holder	1	Y
8mm Linear Shaft	2	N	Bearing Cover	1	Y
Lead Screw Nut	1	N	Y-Axis Carriage	1	Y
8mm Bushing	5	N			

Figure 19 below shows a 3D model of the BSS on its own.

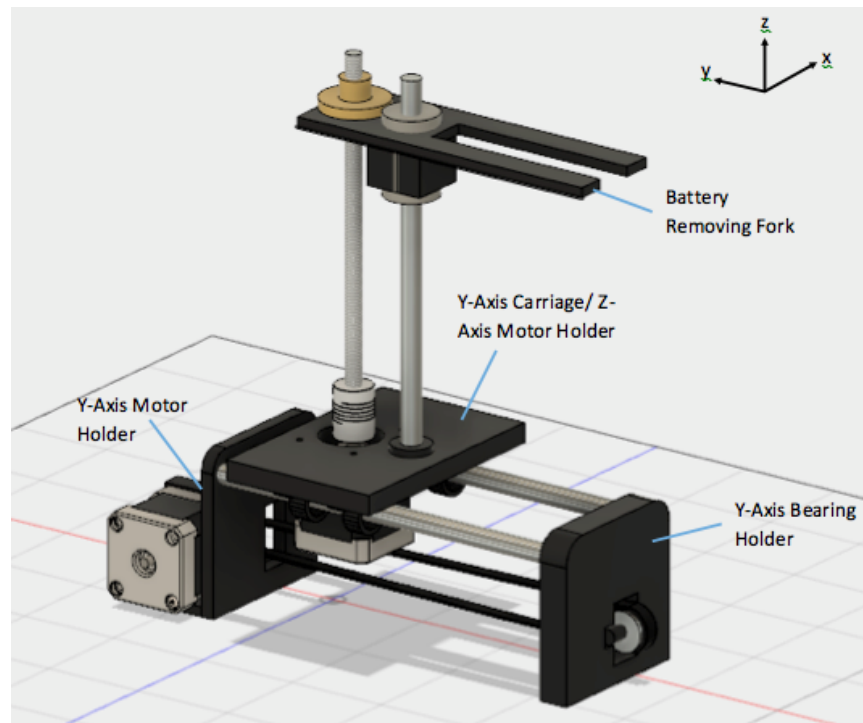


Figure 19. 3D model of the BSS [10] [23].

Figure 20 below shows a complete 3D model of the station with the BSS and ALS.

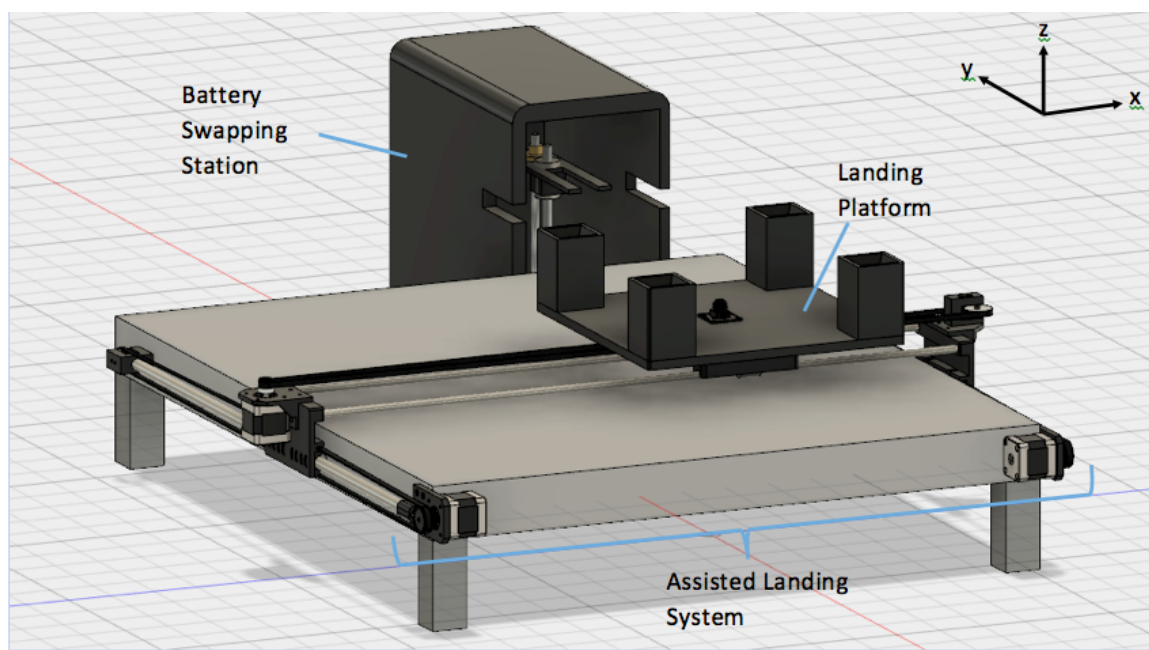


Figure 20. Complete 3D model [10] [11] [23].

As can be seen from the figure above, the forklift system is tucked off to the corner of the station, so as to maximize the landing area and ensure the “Size of Landing Area” spec of 50cm x 50cm from Table 3 is still met. That is where the ALS brings the drone to, so the swapping sequence can begin. In essence, the BSS uses one stepper motor to move the fork in and out (in the y-direction) and another to move the fork up and down (in the z-direction). The BSS inserts its forks into the slots on the battery compartment. The fork then lowers to pull the battery off the drone. The depleted battery is then inserted into a storage unit to be charged. Note that the battery charging is outside the scope of this project, however, the assumption is that the batteries begin to charge once placed into the storage unit. The BSS will then pick up a fully charged battery, and lift up the new battery to the drone. The use of a magnetic connection between the battery and the drone allows for a simple way to connect the new battery. The fork then pulls out, and the drone can then take off with a replaced battery. The microcontroller dictates the fork’s movement by controlling the motors within predetermined limits. The software that controls these movements re-uses the Plane2D class explained in Section 3.2.1.1.

3.4.2 Engineering Design Process

We considered several alternatives during the design process for the BSS. Key factors that affected our final decision on which system to proceed with included (in order of importance) reliability, scalability, and simplicity. Some secondary influential factors were speed, noise, and aesthetics. Accuracy is an extremely important factor, as our system is designed specifically to integrate with our BMS and our custom drone footprint. The assumption is that our design would become the standard that drone companies would adhere to when designing their drones if they wanted to use Swap-it.

The contending designs for this system were a carousal design, a conveyer belt design, and the chosen forklift design. The carousal design used a rotating battery holder, which was placed underneath the drone on the landing platform. When the drone landed, the carousal would rotate pulling the battery off the drone, and immediately locating a new battery under the drone. This design was very quick in swapping the battery, however it had many complications, which led to rejecting the design. It would need to be large in order to hold numerous batteries simultaneously, but also small enough to fit under the landed drone. This would reduce scalability in holding many batteries. Also, it was very difficult to design a rotating mechanism in a small area. This design also put high lateral strain on the drone. In order to remove the battery from the magnetic hold, a force greater than 30N would be required to overcome the magnetic force. This amount of lateral force would be far too much on the drone as laterally there is very little structural support. The second design was a more simplistic method, which used a conveyer belt idea.

The conveyer belt would sit under the drone landing area similar to the carousal, but instead of the rotational motion linear motion is used. The design removes the complexity of the rotational force, but doesn’t resolve the lateral force issue. The forklift system delivers where these other two methods fall short. The forklift system uses vertical force on the drone to remove the battery, which can be much stronger than any lateral force. Pulling down on the drone goes against gravity, keeping a sturdy system unlike the lateral force. The forklift also has a more secure connection to the battery because of the forks that slide into the battery, making the system more reliable.

4 Discussion and Project Timeline

4.1 Evaluation of Final Design

The mechanical designs on the ALS have been fully implemented and initial testing shows the specifications are met. The ALS linear rail system provides minimal friction by the use of the bearings and linear rods, allowing smooth and accurate movements of the platform well beyond the specification. This also allows for very fast movements to meet the specifications on speed. Utilizing the sides of the base for mounting parts has also provided greater landing space. The IPA software has proved to be a performant and stable algorithm, detecting the drone with a very low latency and sending correct coordinates for use by the ALS system. The ALS software has been very reliable as well. Configuration of the ALS software has been somewhat a tedious task but as we gain more experience, this has been cause of little concern. The BSS system has not yet been built, but using the experience of building the ALS and careful designing to meet the specifications, there should be no issues achieving all the desired specifications.

4.2 Use of Advanced Knowledge

Due to the mechanical nature of our project, the majority of the design and analysis requires upper-year Mechanical Engineering (ME) knowledge. However, there is a great deal of upper-year Electrical & Computer Engineering (ECE) knowledge incorporated into the design as well. Namely, the advanced Computer Engineering (CE) knowledge required for the design of the IPA, and the advanced Electrical Engineering (EE) knowledge required for the BMS. Interfacing the microcontroller and stepper motor drivers require a significant deal of advanced ECE knowledge as well.

Table 8 below summarizes the advanced knowledge used in each of the four designed subsystems described in Section 3 above.

Table 8. Summary of advanced knowledge used.

Subsystem	Advanced knowledge used in design and analysis
ALS	• Advanced mechanical design (ME 555) • Mechanical torque, forces, and rotations • Software architecture and maintenance (ECE 452) • Stepper motor driver interfacing and control
IPA	• Image processing in software (ECE 417) • Access protocols and execution analysis in embedded systems (ECE 455) • Microcontroller programming and interfacing
BSS	• Advanced mechanical systems and components • Mechanical and magnetic forces, torque, and rotations • Microcontroller programming (ECE224) • Stepper motor driver interfacing and control

BMS	• PCB design (ECE 473) • Electronic circuit design (ECE 444) • Power management (ECE 463)
-----	---

4.3 Creativity, Novelty, Elegance

4.3.1 Creativity

Swap-it is a creative project in several ways. All four major components (described in section 3 above) of the overall system require a certain degree of creativity. However, the BSS design is where the team needed to be the most creative, as the station is completely responsible for the battery swapping with no help from the drone.

4.3.2 Novelty

Swap-it's novelty can't be attributed to a particular part of the design, but to the overall design. As a whole, Swap-it is an original product that has no competitor currently on the market. To create an autonomous drone battery swapping station that assists the drone's landing while maintaining power to the drone the entire time, opens up a world of possibilities for electric drone use.

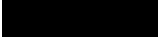
4.3.3 Elegance

Elegance has been an important aspect to the team since the beginning of the design phase. The most elegant parts of the swapping process can be attributed to the ALS and BSS. The ALS seemingly tracks and follows the drone to provide a perfect landing, while the BSS swaps the battery in a smooth and efficient manner that is quite the attention grabber.

4.4 Student Hours

Table 9 below lists how many working hours each team member has contributed to the project as of June 30, 2016.

Table 9. Summary of contributed project hours.

Team Member	Hours to Date
	97
	111
	85.25
	85.25
	59

4.5 Potential Safety Hazards

The most obvious and significant safety hazard is the drone flying around as it lands and takes off again. The flight hazard pertains to both the people around, the drone itself, and the associated parts. To ensure everyone's safety, the drone and Swap-it will be isolated in a meshed off section. The team will get lots of practise flying and landing the drone before adding the BMS to decrease the risk of damaging parts. Moreover, the station will contain several motors and moving parts that could prove hazardous if someone puts their hand in the moving area or drops something in. The motion is also hazardous to the equipment if something were to jam, or if a spatial limit isn't set properly in the software. The use of a power supply and electronic control boards introduces some electrical hazards. The power supply has exposed 120 VAC input terminals, and the microcontroller and MCB circuitry is exposed as well. We are going to cover all exposed terminals and circuitry with plastic guarding to mitigate all electrical hazards. The last potential hazard is a combination of electrical and moving parts. Swap-it requires wires to be run along the moving parts. To prevent the wires from being stretched or crimped, strain relief will be placed where necessary and the wires will be secured in such a way to ensure they can safely reach the furthest and closest parts on the station.

4.6 Project Timeline

Figure 21 below shows a timeline of the tasks involved in our project and their expected order. Over the fall break, we expect to reflect on the feedback provided and make minor improvements on the design.

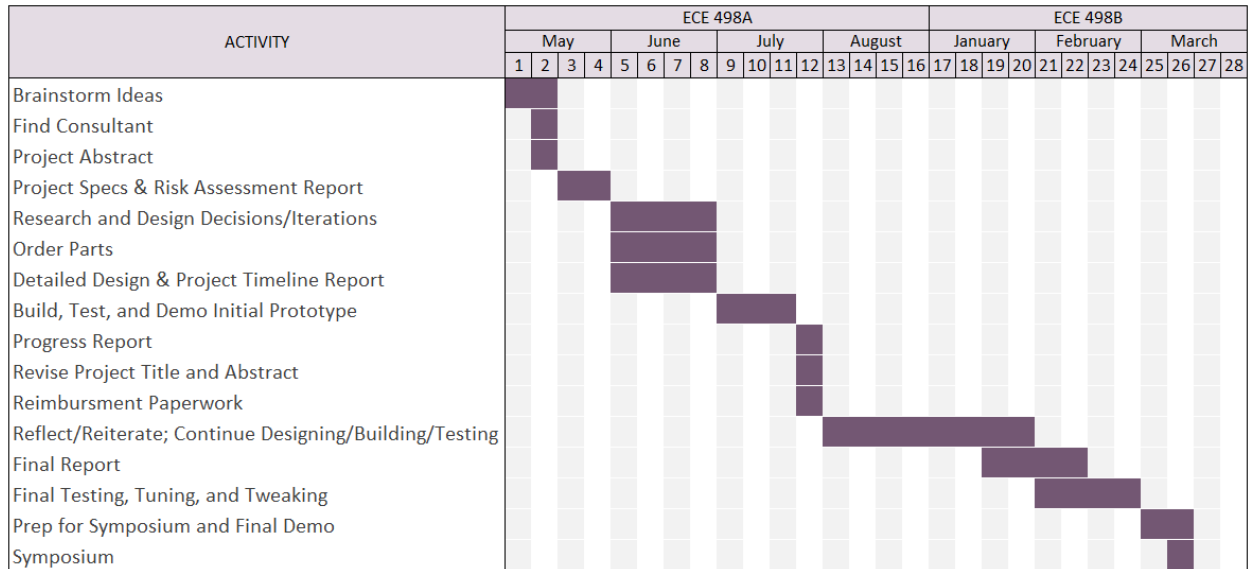


Figure 21. Gantt chart displaying Project Timeline.

References

- [1] M. R. Roberts, "5 drone technologies for firefighting," 20 March 2014. [Online]. Available: <http://www.firechief.com/2014/03/20/5-drone-technologies-firefighting/>.
- [2] E. Howell, "Opportunity Mars Rover Marks 12 Years on Red Planet," 26 January 2016. [Online]. Available: <http://www.space.com/31735-opportunity-rover-12-years-mars.html>.
- [3] B. Carte, "RC Drones Review," 2016. [Online]. Available: <http://rc-drones-review.toptenreviews.com/>.
- [4] OpenCV, "OpenCV API Reference," [Online]. Available: <http://docs.opencv.org/2.4/modules/refman.html>. [Accessed 17 June 2016].
- [5] OpenCV, "Contour Features," [Online]. Available: http://docs.opencv.org/3.1.0/dd/d49/tutorial_py_contour_features.html#gsc.tab=0. [Accessed 17 June 2016].
- [6] pyimagesearch, "Ball Tracking with OpenCV," [Online]. Available: <http://www.pyimagesearch.com/2015/09/14/ball-tracking-with-opencv/>. [Accessed 16 June 2016].
- [7] OpenCV, "Morphological Transformations," [Online]. Available: http://docs.opencv.org/3.0-beta/doc/py_tutorials/py_imgproc/py_morphological_ops/py_morphological_ops.html. [Accessed 16 June 2016].
- [8] B. Gregg, "Flame Graphs," 15 June 2016. [Online]. Available: <http://www.brendangregg.com/flamegraphs.html>.
- [9] Autodesk, "Fusion 360 Overview," 2016. [Online]. Available: <http://www.autodesk.com/products/fusion-360/overview>. [Accessed 20 June 2016].
- [10] GrabCAD Community, "Library," [Online]. Available: <https://grabcad.com/library/stepper-motor-nema-17>. [Accessed 20 June 2016].
- [11] JJ Robots, "Air Hockey Robot (a 3D printer hack)," [Online]. Available: <https://jjrobots.com/air-hockey-robot-a-3d-printer-hack/>. [Accessed 22 June 2016].
- [12] GitHub, "laurb9/Stepper Driver," [Online]. Available: <https://github.com/laurb9/StepperDriver>. [Accessed 13 June 2016].

GitHub, "MarlinFirmware/Marlin," [Online]. Available: <https://github.com/MarlinFirmware/Marlin>.
[13] [Accessed 14 June 2016].

GitHub, "mmarchetti/DirectIO#benchmarks," [Online]. Available:
[14] <https://github.com/mmarchetti/DirectIO#benchmarks>. [Accessed 19 June 2016].

Hobby People, "hpheli-f450-quad-diy-starter-kit," [Online]. Available:
[15] <http://www.hobbypeople.net/index.php/hpheli-f450-quad-diy-starter-kit.html>. [Accessed 20 June 2016].

Fairchild Semiconductor, "FDS6574A 20V N-Channel PowerTrench Mosfet," May 2008. [Online].
[16] Available: <https://www.fairchildsemi.com/datasheets/FD/FDS6574A.pdf>.

Hobby People, "hpheli-motor-2212-bl-for-multi-rotor-1000kv," [Online]. Available:
[17] <http://www.hobbypeople.net/index.php/hpheli-motor-2212-bl-for-multi-rotor-1000kv.html>.
[Accessed 20 June 2016].

Linear Technology, "Low Voltage Ideal Diode Controller with Monitoring," [Online]. Available:
[18] <http://cds.linear.com/docs/en/datasheet/4352fa.pdf>. [Accessed 18 June 2016].

Texas Instruments, "Advanced Linear Charge Management IC for Single and Two Cell Lithium-ion
[19] and Lithium-polymer," May 2001. [Online]. Available:
<http://www.ti.com/lit/ds/symlink/bq2057w.pdf>.

Magcraft, "Pull Force Calculator," [Online]. Available: [http://www.rare-earth-magnets.com/pull-](http://www.rare-earth-magnets.com/pull-force-calculator)
[20] [force-calculator](http://www.rare-earth-magnets.com/pull-force-calculator). [Accessed 18 June 2016].

Cirris Systems, "Wire Resistance Calculator and Table," [Online]. Available:
[21] <http://www.cirris.com/learning-center/calculators/133-wire-resistance-calculator-table>. [Accessed 19 June 2016].

Wikipedia, "Rare earth magnet," 24 April 2016. [Online]. Available:
[22] https://en.wikipedia.org/wiki/Rare-earth_magnet.

GrabCAD, "Library - 5mm-to-8mm-flexible-shaft-coupler-1," [Online]. Available:
[23] <https://grabcad.com/library/5mm-to-8mm-flexible-shaft-coupler-1>. [Accessed 12 June 2016].